

Distributed Transactions

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles

Phase 2 Chapter 11 flagged, without fully resolving, that splitting a monolith into microservices breaks the single-database ACID transaction you'd otherwise take for granted. This chapter resolves it properly: how do you make an operation that spans multiple services (and multiple databases) behave correctly, given everything Chapters 1–3 established about partial failure and the cost of consensus?

A concrete motivating example: placing an order might need to (1) reserve inventory in the inventory service, (2) charge the customer in the payments service, and (3) create a shipment record in the logistics service. In a monolith with one database, this is a single ACID transaction — all three succeed together, or all three roll back together, guaranteed. Across three separate services with three separate databases, there is no single database transaction that can span all three, so a different mechanism is needed to keep them consistent.

Two-Phase Commit (2PC): the strongly-consistent, blocking approach

Two-Phase Commit is the classic approach to distributed transactions, using a central **coordinator** and two rounds of communication. In the **prepare phase**, the coordinator asks every participant (each service/database involved) "can you commit this?" — each participant does whatever work is needed to guarantee it *can* commit if asked (locking rows, writing to a durable log) and replies yes or no, without actually committing yet. Only if every participant replies yes does the coordinator move to the **commit phase**, telling everyone to actually commit; if any participant replied no (or timed out), the coordinator tells everyone to abort instead.

2PC gives you the strong, all-or-nothing guarantee you're used to from a single-database transaction, but at a real cost that becomes clear once you connect it to Chapter 1's partial-failure problem: every participant must hold its locks (blocking other operations on that data) for the *entire* duration of both phases, and if the coordinator itself crashes after the prepare phase but before sending the commit decision, participants are left **blocked** indefinitely, unable to safely commit or abort on their own, since they don't know what the coordinator ultimately decided. This blocking behavior, combined with the coordinator being a single point of failure, is why 2PC scales poorly and is largely avoided in modern high-throughput microservices architectures, despite being the theoretically "correct" and simplest-to-reason-about approach.

Backend engineer analogy

Imagine coordinating a group dinner reservation across three friends' calendars by phone: you call each friend and ask them to tentatively hold that evening (prepare), and only once all three confirm do you call back to finalize (commit). If you, the coordinator, get hit by a bus between the tentative holds and the final confirmation, all three friends are stuck holding an indefinite tentative block on their calendar, unable to book anything else, not knowing if the dinner is actually happening. That's exactly 2PC's blocking failure mode.

Saga pattern: the eventually-consistent, non-blocking approach

The **Saga pattern** takes a fundamentally different approach: instead of one atomic cross-service transaction, break the operation into a sequence of independent **local transactions**, each fully committing within its own service's database immediately, with each step publishing an event that triggers the next step. If a step fails partway through, instead of a global rollback, the saga runs **compensating transactions** — explicit, custom-written operations that semantically undo each already-completed step ("release the inventory reservation," "refund the payment") in reverse order.

Sagas can be coordinated in two ways. **Choreography** is decentralized: each service listens for events from the previous step and independently decides what to do next, with no central coordinator — simpler for small sagas, but the overall flow becomes hard to see in one place as the number of steps grows, since the logic is scattered across every participating service. **Orchestration** uses a central orchestrator service that explicitly calls each step and explicitly triggers compensations on failure — easier to understand, monitor, and debug as a single, visible flow, at the cost of that orchestrator becoming a new, important piece of infrastructure in its own right.

The trade-off between 2PC and Sagas is a direct, practical instance of Phase 2 Chapter 4's CAP/PACELC framework: 2PC chooses consistency and blocks (unavailable) during coordinator failure or slow participants; Sagas choose availability and accept a temporary window of inconsistency (an order that's "reserved" but not yet "paid") that resolves eventually, either forward to completion or backward via compensation. Virtually all modern, high-throughput microservices systems choose Sagas over 2PC for this exact reason — the availability and throughput cost of 2PC's blocking behavior is rarely worth its stronger guarantee, especially since idempotency (Phase 2 Chapter 9) and careful compensation design can make Sagas reliable enough for even financial operations.

One nuance worth being precise about: compensating transactions are not a true database rollback — they're a new, forward-moving operation that semantically reverses an earlier one. "Refund the payment" is a new transaction, not an undo of the original charge, which matters because the original charge and the refund are both visible, auditable events, and because compensations themselves need to be idempotent and can, in rare cases, themselves fail and need their own handling — a genuine source of design complexity that a senior engineer should name directly rather than gloss over.

2. Real-World Example / Case Study

Swiggy's and Zomato's order-placement flow — reserve inventory at the restaurant, charge the customer, notify the restaurant, assign a delivery partner — is a textbook Saga: if payment fails after inventory was tentatively reserved, a compensating action releases that reservation rather than the system attempting a single atomic cross-service transaction across four separate services.

Razorpay's and PhonePe's merchant settlement flows, which touch the payment ledger, a merchant's payout account, and potentially a tax/compliance record, are commonly implemented as orchestrated Sagas precisely because a central, visible orchestrator makes it far easier to audit exactly what happened during a partial failure — a genuine requirement in a regulated financial domain where compensating actions need clear, traceable records.

A Careem or Talabat multi-service booking flow (reserve driver, charge fare, notify rider) similarly avoids 2PC's blocking cost — at ride-hailing volume, holding cross-service locks for the duration of a coordinator round-trip would create unacceptable latency and contention during peak hours, making the

Saga pattern's non-blocking, eventually-consistent approach the practical necessity, not just a theoretical preference.

3. Diagram

Two-Phase Commit (blocking):
Coordinator -- PREPARE --> [Inventory] [Payment] [Shipping]
Coordinator <-- YES/YES/YES --
Coordinator -- COMMIT --> [Inventory] [Payment] [Shipping]
(If coordinator crashes between PREPARE and COMMIT: all 3 participants are BLOCKED, holding locks, unable to proceed)
Saga (orchestrated, non-blocking):
[Orchestrator] -> reserve inventory -> OK
[Orchestrator] -> charge payment -> FAILS
[Orchestrator] -> runs COMPENSATION: release inventory reservation
Result: no global lock ever held; failure handled by a new, forward-moving compensating transaction, not a rollback

4. Trade-offs Table

Approach	Consistency	Availability / Throughput	Complexity
Two-Phase Commit (2PC)	Strong, all-or-nothing	Poor — blocking, coordinator SPOF	Conceptually simple, operationally fragile
Saga (choreography)	Eventual, via compensation	High — no cross-service locks	Flow scattered across services, hard to trace
Saga (orchestration)	Eventual, via compensation	High — no cross-service locks	Central orchestrator adds a new component, but flow is visible

5. Common Interview Questions

Q1: "Why does 2PC scale poorly in practice?"

Model approach: Explain the blocking behavior directly — participants hold locks for the full duration of both phases, and if the coordinator crashes mid-protocol, participants are stuck unable to safely commit or abort, creating both a latency and an availability problem at real throughput.

Q2: "Design the Saga for a ride-booking flow: reserve driver, charge fare, notify rider. What happens if charging the fare fails?"

Model approach: Walk through each local transaction step, then describe the compensating action specifically for the failure point — releasing the driver reservation — and note that steps already completed (driver reserved) are compensated, not force-rolled-back at the database level.

Q3: "Choreography or orchestration for a 6-step saga? Justify your choice."

Model approach: Recommend orchestration for a saga with more than a handful of steps, explaining that choreography's scattered event-listening logic becomes hard to trace and debug as step count

grows, while an orchestrator keeps the flow visible and centrally auditable, which matters especially for regulated or financial flows.

Q4: "Are compensating transactions the same as a database rollback? Why does this distinction matter?"

Model approach: No — explain that a compensation is a new, forward-moving operation that semantically undoes a prior step (a refund, not an erased charge), which matters for auditability and because compensations themselves need idempotency and can fail, requiring their own handling.

6. Practice Exercises

- 1 Design an orchestrated Saga for an e-commerce checkout (reserve inventory, charge card, create shipment, send confirmation), specifying the compensating action for each step.
- 2 Explain, step by step, exactly how a coordinator crash between the prepare and commit phases leaves 2PC participants blocked, and why they can't safely resolve it themselves.
- 3 Compare choreography vs. orchestration for a 3-step saga vs. an 8-step saga, and justify why your recommendation changes (or doesn't) between the two.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Distributed Transactions Explained: 2 Phase Commit vs Saga Pattern"	Direct side-by-side comparison of both approaches this chapter covers.
English	"Saga Pattern Explained: Distributed Transactions in Microservices"	Focused deep dive into Saga mechanics and compensation design.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers distributed transactions as part of a full bilingual course.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Places 2PC/Saga in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- Splitting a monolith into microservices breaks single-database ACID transactions; a new mechanism is needed.
- Two-Phase Commit gives strong, all-or-nothing consistency but blocks participants and has a coordinator single point of failure.
- The Saga pattern breaks an operation into local transactions per service, using compensating transactions to handle failure.
- Choreography (decentralized) suits small sagas; orchestration (centralized) suits larger, more auditable flows.
- Compensating transactions are new, forward-moving operations, not database rollbacks, and need their own idempotency.

- Most modern microservices systems choose Sagas over 2PC, trading strong consistency for availability and throughput.
- This is a direct, practical application of Phase 2 Chapter 4's CAP/PACELC consistency-vs-availability trade-off.